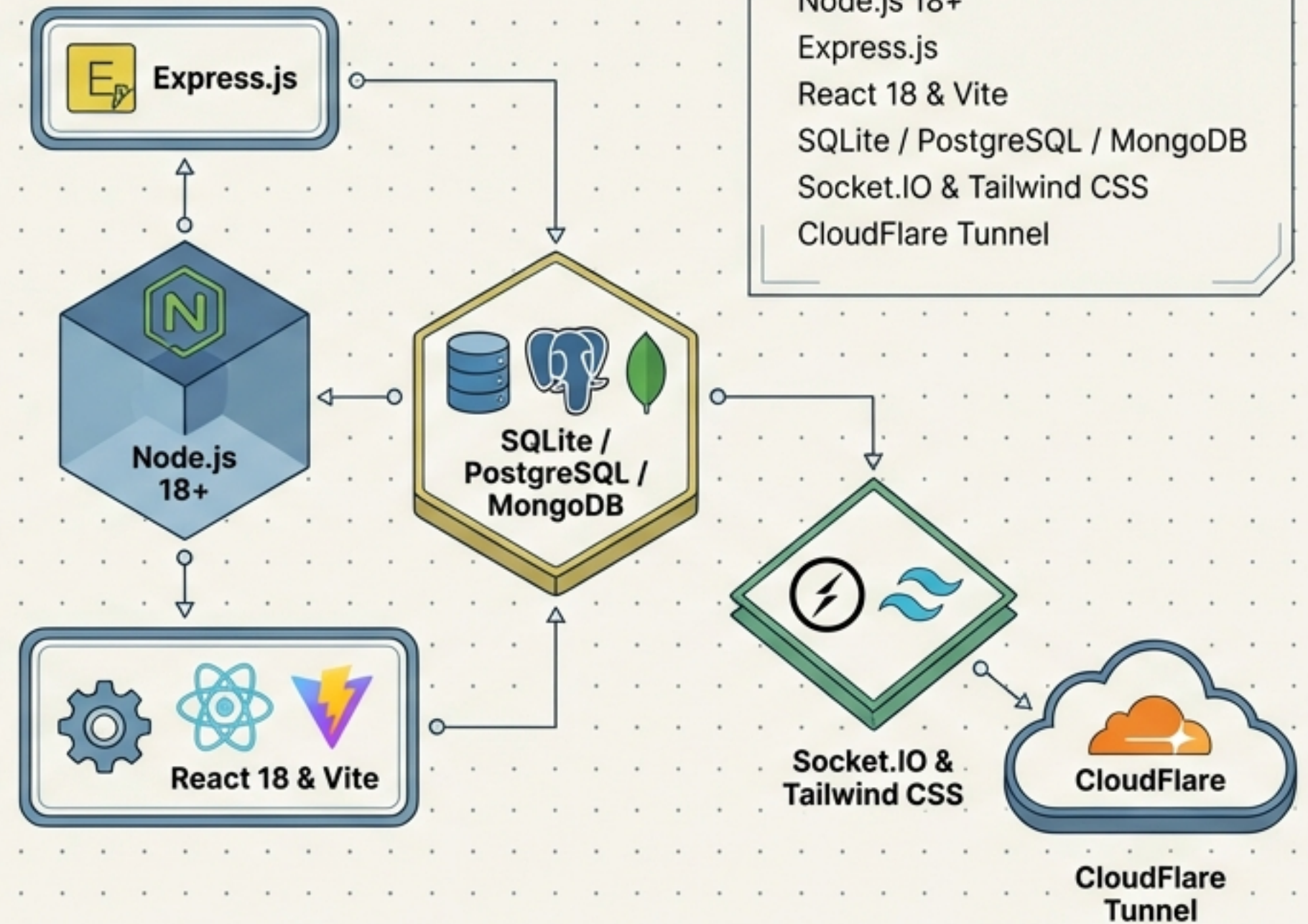


Modul 05

Web Server & Full-Stack JavaScript

Membangun Aplikasi Modern dari Backend hingga Deploy



Tujuan Pembelajaran: Mahasiswa mampu merancang, membangun, dan men-deploy aplikasi web full-stack secara mandiri.

THE PREMIUM ENGINEERING MANUAL: JavaScript Full-Stack Pipeline



Satu Ekosistem

JavaScript mengontrol logika frontend dan backend tanpa konteks switching.



Format Universal

Data mengalir dalam wujud JSON asli tanpa perlu konversi rumit antar layer.



Performa Maksimal

Event loop Node.js menangani concurrent requests dengan non-blocking I/O secara efisien.



THE PREMIUM ENGINEERING MANUAL: Node.js Fundamentals & NPM

Runtime V8 Chrome di sisi server. Single-thread dengan eksekusi non-blocking I/O.
Didukung oleh ekosistem raksasa npm (2 juta+ *packages*).

Sistem Modul

CommonJS (Legacy)	ESM (Modern)
<code>require()</code> <code>module.exports</code>	<code>import</code> <code>export</code>

```
> node --version
v18.16.0
> npm init -y
Wrote to /Users/developer/project/package.json

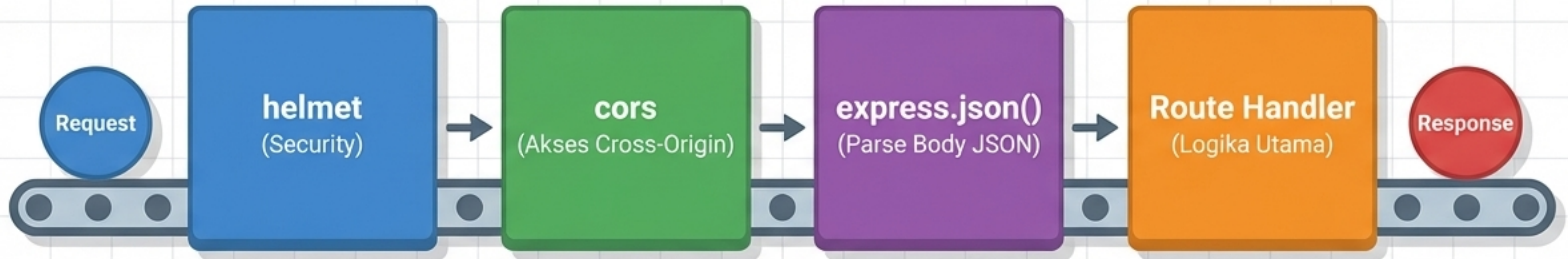
> npm install express
added 60 packages, and audited 61 packages in 2s
> npm run dev

> project@1.0.0 dev
> nodemon index.js

[nodemon] 2.0.22
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,json
[nodemon] starting `node index.js`
Server running at http://localhost:3000
```

THE PREMIUM ENGINEERING MANUAL:

Express.js Setup & Middleware Chain



```
npm install express nodemon cors helmet morgan

const express = require('express');
const app = express();

app.use(helmet());
app.use(cors());
app.use(express.json()); // Wajib untuk parse JSON body
```

Middleware adalah fungsi berantai yang memproses request sebelum mencapai route handler.

THE PREMIUM ENGINEERING MANUAL:

Express Routing & REST Verbs



GET

Membaca data



POST

Membuat data baru (201)



PUT

Memperbarui data



DELETE

Menghapus data (204)

```
1  const express = require('express');
2  const router = express.Router();
3
4
5  // Menangkap Parameter
6  const id = req.params.id;
7  const sort = req.query.sort;
8
9
10 // Error Middleware (Wajib 4 Parameter)
11 app.use((err, req, res, next) => {
12   console.error(err.stack);
13
14   res.status(500).send('Terjadi kesalahan!');
15 });
```

Dari URL:
/api/todos/123

Dari URL:
?sort=asc

THE PREMIUM ENGINEERING MANUAL:

HTTP Status Codes: Kamus Komunikasi Server

Diagnostic Table

✓ Status Sukses	! Status Error
✓ 200 OK: Request berhasil diproses	✗ 400 Bad Request: Validasi input gagal
✓ 201 Created: Data baru berhasil dibuat (POST)	✗ 401 Unauthorized: Belum login / tidak ada token
✓ 204 No Content: Berhasil, tidak ada data dikembalikan (DELETE)	✗ 403 Forbidden: Tidak memiliki hak akses
	✗ 404 Not Found: Data atau endpoint tidak ditemukan
	✗ 500 Server Error: Kesalahan internal backend

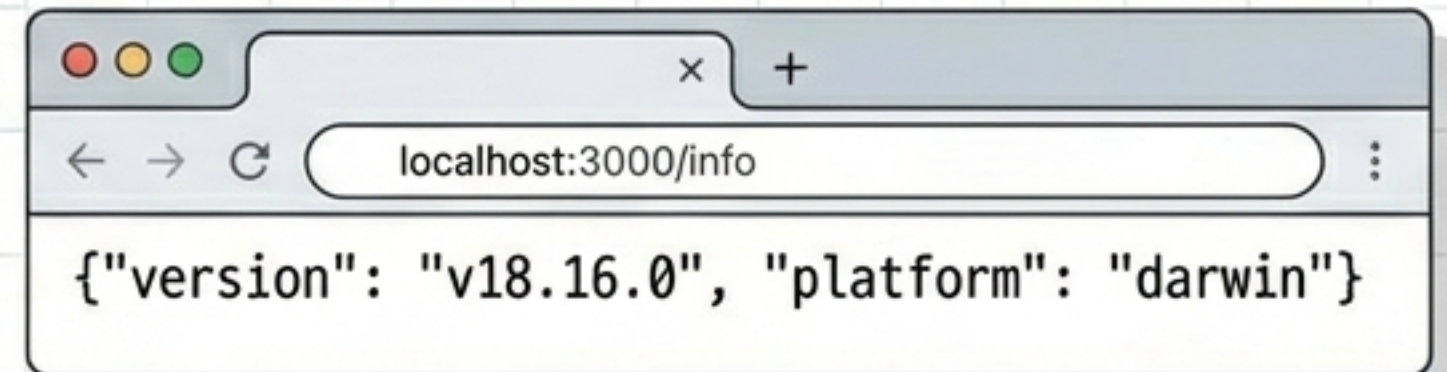
```
// Best Practice: Berikan status yang tepat agar frontend mudah menangani error
res.status(404).json({ success: false, error: 'Data tidak ditemukan' });
```

THE PREMIUM ENGINEERING MANUAL:

Percobaan 1 – Express Hello World

```
> mkdir p1 && cd p1
> npm init -y
> npm install express nodemon
> npm run dev
```

```
[nodemon] starting `node server.js`
[nodemon] starting `node server.js`
Server berjalan di http://localhost:3000
```



Observasi: Nodemon akan otomatis me-restart server setiap kali file disimpan.

THE PREMIUM ENGINEERING MANUAL:

REST API Design Pattern

Respons Mentah (Dihindari)



```
[  
  { id: 1, title: 'Test' }  
]
```

Respons Terstruktur (Standar)



```
{  
  success: true,  
  data: [ { id: 1, title: 'Test' } ],  
  count: 1  
}
```

URL Jamak Konsisten

Gunakan /api/todos (bukan /api/getTodo).

Validasi Input

Gunakan express-validator:
body('title').notEmpty()

Async Error Wrapper

```
const wrap = fn => (req, res, next) =>  
  Promise.resolve(fn(req, res, next)).catch(next);
```

THE PREMIUM ENGINEERING MANUAL:

Percobaan 2 – CRUD In-Memory

GET

http://localhost:3000/api/todos?done=true

Send

Query Params

	Key	Value	Description	...	Bulk Edit
☒	done	true			

Response 200 OK

```
[  
  {  
    id: 2,  
    title: 'Belajar Express',  
    done: true  
  }  
]
```

GET /api/todos

GET /api/todos/:id

POST /api/todos (201)

PUT /api/todos/:id

DELETE /api/todos/:id (204)



Peringatan Volatilitas!

Menggunakan Array JavaScript (`let todos = []`) berarti data in-memory akan lenyap seketika saat server di-restart. Kita butuh database permanen.

THE PREMIUM ENGINEERING MANUAL: SQLite dengan better-sqlite3



```
npm install better-sqlite3
```

```
const Database = require('better-sqlite3');  
const db = new Database('todos.db');
```

```
db.exec(`CREATE TABLE IF NOT EXISTS todos (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  title TEXT,  
  done INTEGER DEFAULT 0  
)`);
```

.run()



Operasi **Write (INSERT / UPDATE / DELETE)**.
Mengembalikan objek dengan **lastInsertRowid**.

.get()



Operasi **Read One (SELECT WHERE)**.
Mengembalikan 1 objek baris tunggal.

.all()



Operasi **Read Many (SELECT ALL)**.
Mengembalikan array berisi banyak objek.

API Synchronous yang sangat cepat, nol konfigurasi, dan file .db langsung tersimpan di dalam folder project.

THE PREMIUM ENGINEERING MANUAL: Percobaan 3 – Integrasi SQLite Backend



```
// Menambah Data Baru
const stmtInsert = db.prepare('
  INSERT INTO todos (title) VALUES (?)
');
stmtInsert.run(req.body.title);
```



```
// Mengambil Semua Data
const stmtAll = db.prepare('
  SELECT * FROM todos
');
const data = stmtAll.all();
```



```
[nodemon] restarting due to changes...
[nodemon] starting node server.js
Server berjalan di port 3000
Database terhubung.
```



project-folder



server.js



package.json



todos.db (Data Persisten)

THE PREMIUM ENGINEERING MANUAL:

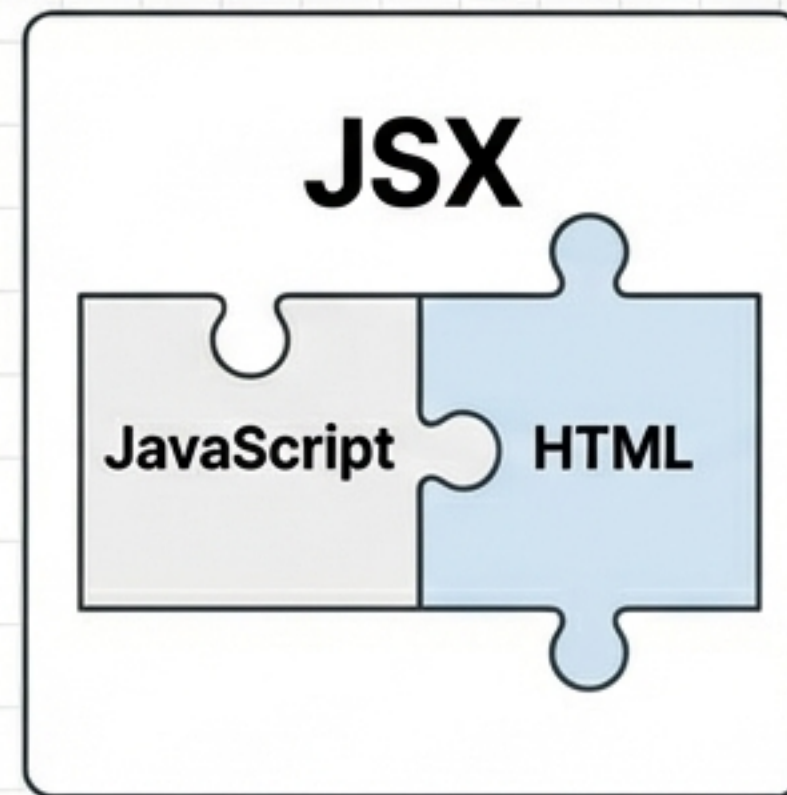
React + Vite Setup



```
npm create vite@latest app --template react
cd app && npm install
npm run dev // Berjalan di localhost:5173
```

app/

- index.html
- package.json
- src/
 - main.jsx
 - App.jsx (Area Kerja Utama)



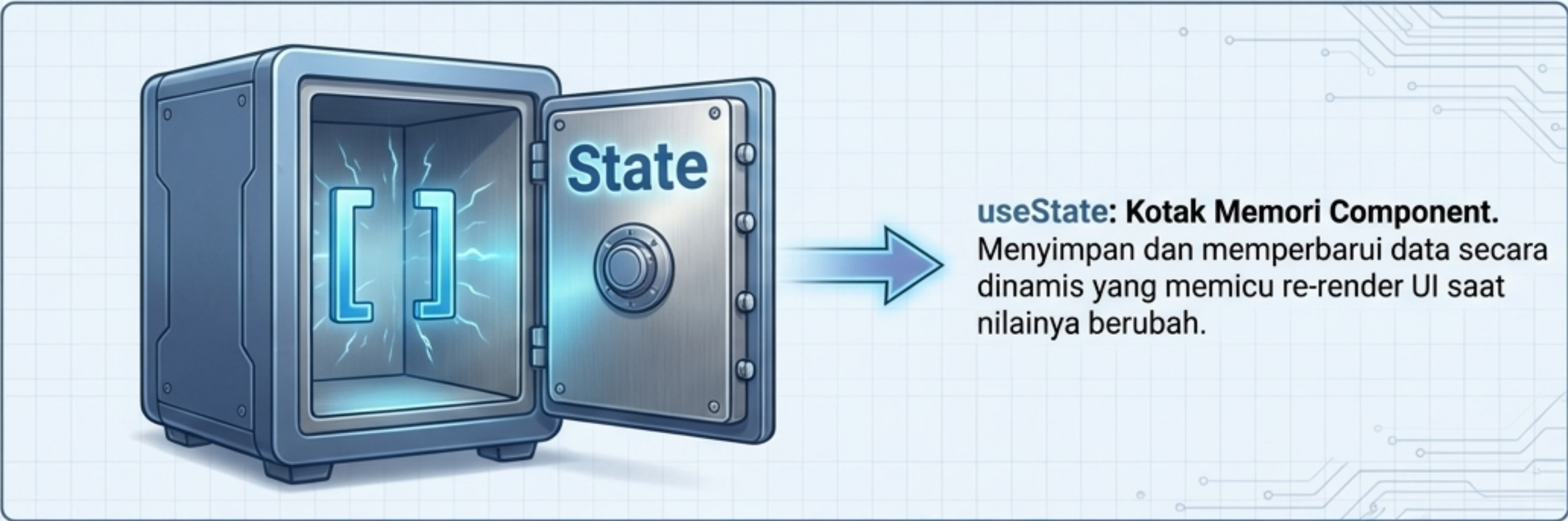
Functional Components

Membangun antarmuka UI secara modular menggunakan blok fungsi JavaScript murni.

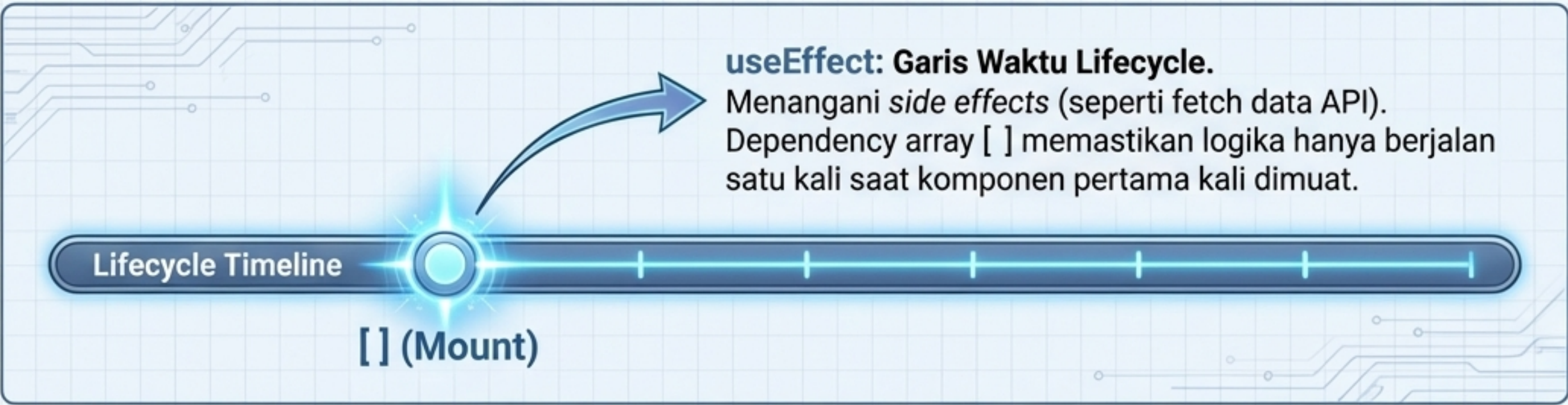
JSX Syntax

Memungkinkan penulisan markup mirip HTML secara langsung di dalam logika JavaScript.

THE PREMIUM ENGINEERING MANUAL: React Hooks Penting: Logika Dinamis Frontend



Visualisasi useState sebagai penyimpanan data.



Visualisasi useEffect sebagai eksekusi berdasarkan waktu.

useRef

Akses langsung ke elemen DOM tanpa memicu re-render.



Custom Hook

Gabungan hooks dasar (contoh: **useFetch**) untuk logika yang dapat digunakan ulang.



THE PREMIUM ENGINEERING MANUAL: Axios untuk HTTP Client Jembatan Data Frontend ke Backend



Native Fetch API (2 Langkah)

```
const res = await fetch('/api/todos');  
const data = await res.json(); // Parse manual  
console.log(data);
```



Axios (1 Langkah & Auto-Parse)

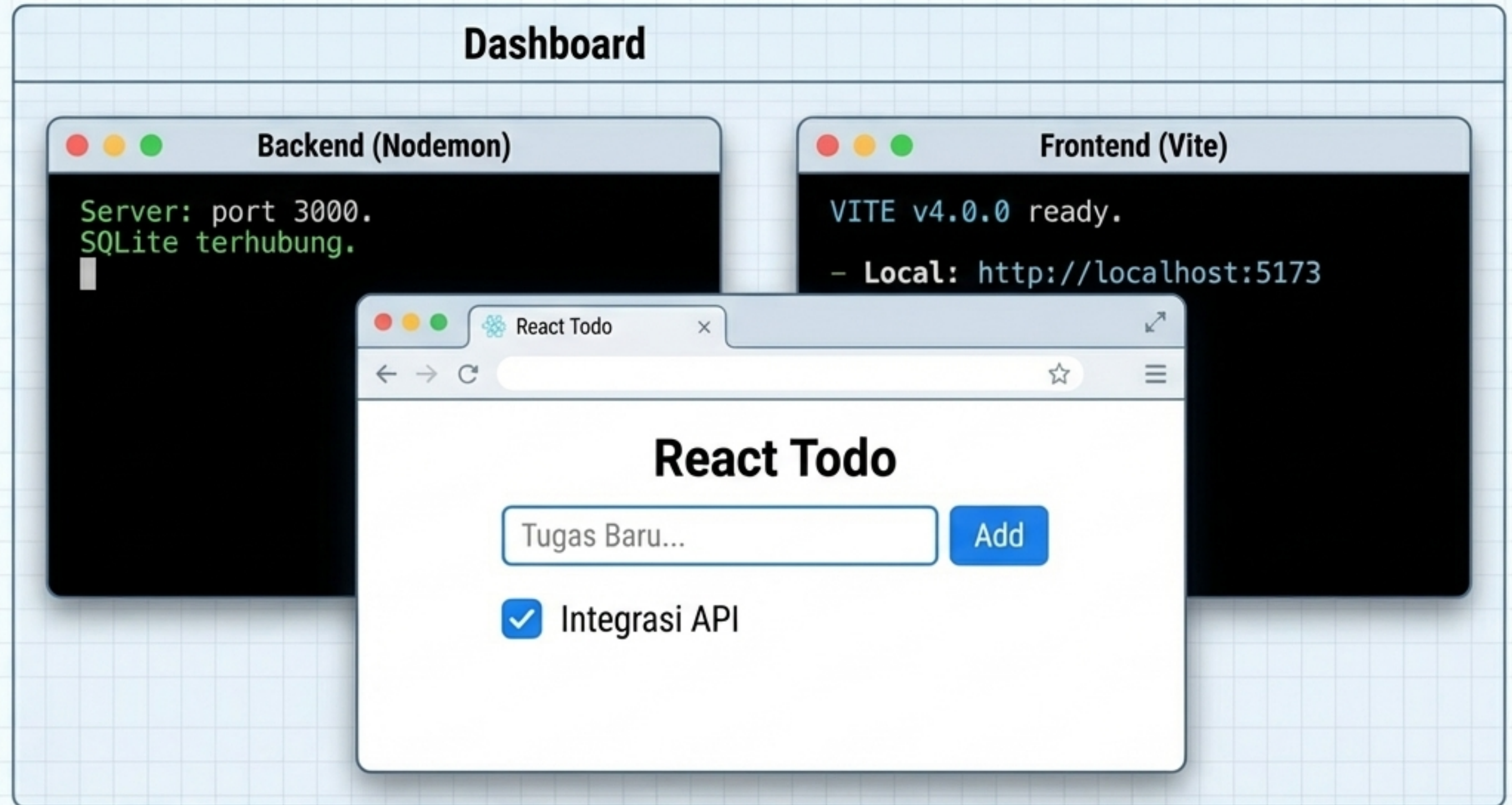
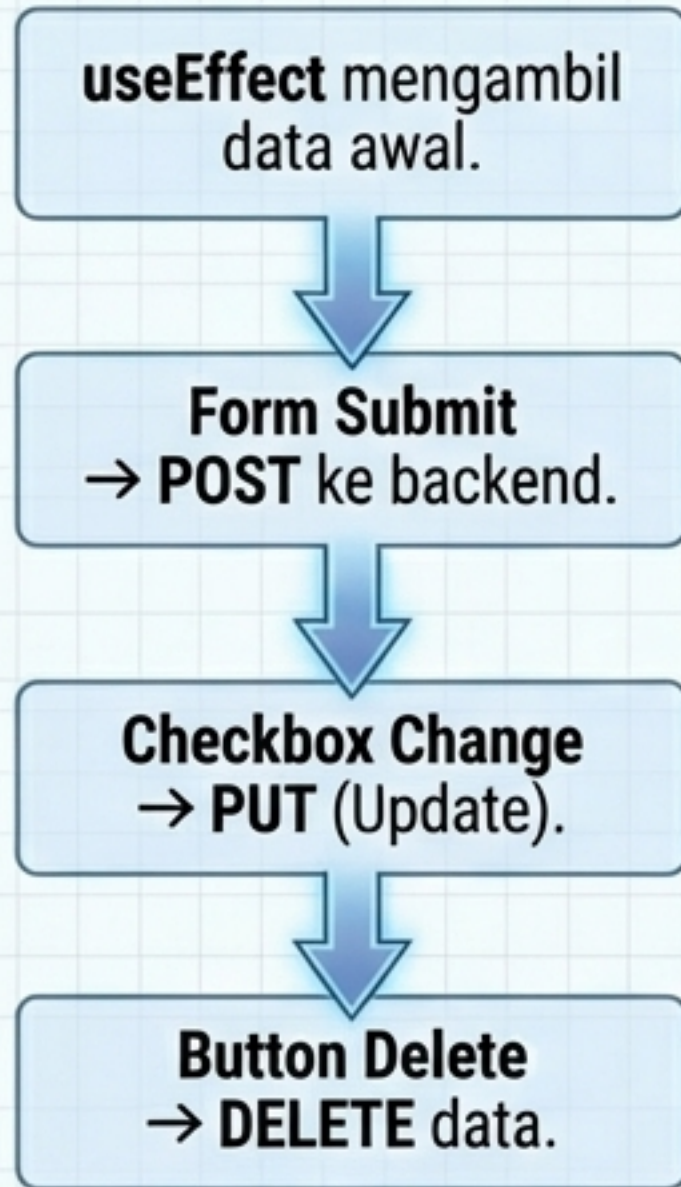
```
const res = await axios.get('/api/todos');  
// res.data sudah berupa payload JSON  
console.log(res.data);
```

```
npm install axios
```

```
const api = axios.create({  
  baseURL: 'http://localhost:3000/api'  
});
```

```
try {  
  const response = await api.post('/todos', { title: 'Tugas Baru' });  
} catch (error) {  
  console.error('Gagal menghubungi server', error);  
}
```

THE PREMIUM ENGINEERING MANUAL: Percobaan 4 – Integrasi Full-Stack



Critical Fix: Kebijakan CORS

Aplikasi React akan diblokir oleh browser kecuali backend mengaktifkan akses. Pastikan **npm install cors** dan **app.use(cors())** telah diterapkan di server.js!